```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int data;
  struct Node* next;
};

struct Node* createNode(int data) {
  struct Node* newNode = malloc(sizeof(struct Node));

  if (newNode == NULL) {
    printf("Memory allocation failed!\n");
    exit(1);
  }

  newNode->data = data;
  newNode->next = NULL;

  return newNode;
}

void display(struct Node* head) {
  struct Node* temp = head;

  if (temp == NULL) {
    printf("List is empty\n");
    return;
  }

  printf("Linked List: ");

  while (temp != NULL) {
    printf("%d -> ", temp->data);
    temp = temp->next;
  }
```

```
    printf("NULL\n");
}

void insertAtBeginning(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    newNode->next = *head;
    *head = newNode;
}

void insertAtEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    struct Node* temp = *head;

    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->next = newNode;
}

void insertAtPosition(struct Node** head, int data, int position) {
    if (position < 1) {
        printf("Invalid position!\n");
        return;
    }

    if (position == 1) {
        insertAtBeginning(head, data);
        return;
```

```
    }

    struct Node* temp = *head;

    for (int i = 1; i < position - 1 && temp != NULL; i++) {
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Position out of range!\n");
        return;
    }

    struct Node* newNode = createNode(data);
    newNode->next = temp->next;
    temp->next = newNode;
}

void deleteAtBeginning(struct Node** head) {
    if (*head == NULL) {
        printf("List is empty!\n");
        return;
    }

    struct Node* temp = *head;
    *head = temp->next;
    free(temp);
}

void deleteAtEnd(struct Node** head) {
    if (*head == NULL) {
        printf("List is empty!\n");
        return;
    }

    if ((*head)->next == NULL) {
```

```
        free(*head);
        *head = NULL;
        return;
    }

    struct Node* temp = *head;

    while (temp->next->next != NULL) {
        temp = temp->next;
    }

    free(temp->next);
    temp->next = NULL;
}

void deleteAtPosition(struct Node** head, int position) {
    if (*head == NULL) {
        printf("List is empty!\n");
        return;
    }

    if (position < 1) {
        printf("Invalid position!\n");
        return;
    }

    if (position == 1) {
        deleteAtBeginning(head);
        return;
    }

    struct Node* temp = *head;

    for (int i = 1; i < position - 1 && temp != NULL; i++) {
        temp = temp->next;
    }
```

```
    if (temp == NULL || temp->next == NULL) {
        printf("Position out of range!\n");
        return;
    }

    struct Node* nextNode = temp->next->next;
    free(temp->next);
    temp->next = nextNode;
}

void freeList(struct Node** head) {
    struct Node* temp;

    while (*head != NULL) {
        temp = *head;
        *head = (*head)->next;
        free(temp);
    }
}

int main() {
    struct Node* head = NULL;
    int choice, data, position;

    while (1) {
        printf("\n--- Linked List Menu ---\n");
        printf("1. Insert at Beginning\n");
        printf("2. Insert at End\n");
        printf("3. Insert at Position\n");
        printf("4. Delete at Beginning\n");
        printf("5. Delete at End\n");
        printf("6. Delete at Position\n");
        printf("7. Display\n");
        printf("8. Exit\n");
```

```
printf("Enter your choice: ");

if (scanf("%d", &choice) != 1) {
    printf("Invalid input!\n");
    freeList(&head);
    return 0;
}

switch (choice) {
    case 1:
        printf("Enter data: ");
        if (scanf("%d", &data) != 1) {
            freeList(&head);
            return 0;
        }
        insertAtBeginning(&head, data);
        break;

    case 2:
        printf("Enter data: ");
        if (scanf("%d", &data) != 1) {
            freeList(&head);
            return 0;
        }
        insertAtEnd(&head, data);
        break;

    case 3:
        printf("Enter data: ");
        if (scanf("%d", &data) != 1) {
            freeList(&head);
            return 0;
        }

        printf("Enter position: ");
        if (scanf("%d", &position) != 1) {
```

```
            freeList(&head);
            return 0;
        }

        insertAtPosition(&head, data, position);
        break;

    case 4:
        deleteAtBeginning(&head);
        break;

    case 5:
        deleteAtEnd(&head);
        break;

    case 6:
        printf("Enter position: ");
        if (scanf("%d", &position) != 1) {
            freeList(&head);
            return 0;
        }

        deleteAtPosition(&head, position);
        break;

    case 7:
        display(head);
        break;

    case 8:
        freeList(&head);
        printf("Program terminated.\n");
        return 0;

    default:
        printf("Invalid choice!\n");
```

```
    }
  }
}
```